BIG DATA, INDIVIDUAL ASSIGNMENT 3

Parallel Matrix Multiplication

How many threads make it faster, and when it does not help

Pablo Castillo Garcerá

Grado en Ciencia e Ingeniería de Datos, 2026-2027

The question: how much faster with more threads?

My CPU has 12 cores, but until now my programs used only one of them.

Two versions, same code

Sequential: the blocked method from Assignment 2, with one thread.

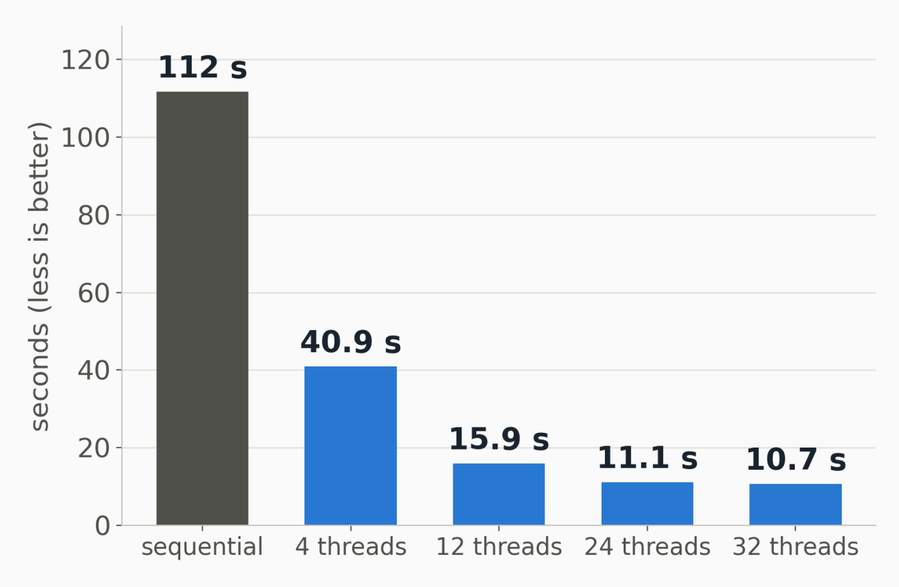
Parallel: the rows of C are cut into chunks, and each thread takes the next free chunk.

The test

Sizes from 256 to 8192, and 1 to 32 threads on 12 cores (24 logical).

The result is exactly the same with every thread count (87 tests).

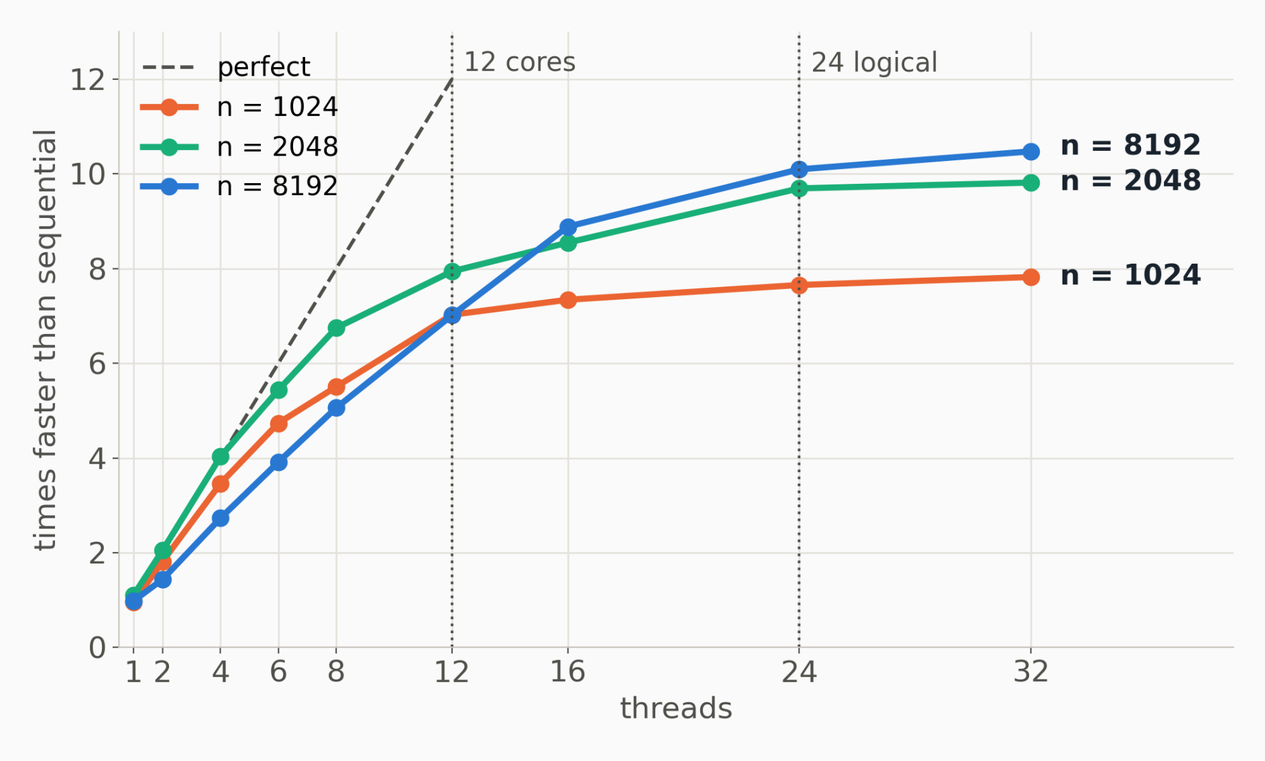
Time for a 8192 × 8192 matrix



From 112 seconds to 11 seconds with 24 threads.

Big matrices: up to 10 times faster

Speedup = sequential time / parallel time. The dashed line is perfect.



× 7 to 8

12 threads, one for each core. Not 12: they share the memory, and some parts use one thread.

× 10

24 threads: each core runs two threads at the same time.

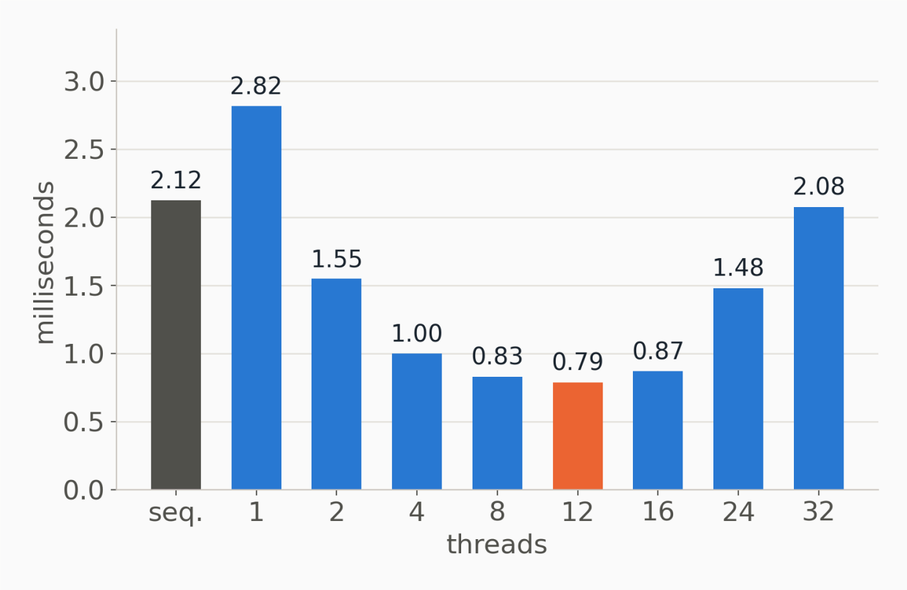
+ 4% only

32 threads: more than the CPU can run, so Windows switches between them.

Small matrices: the threads cost more than the work

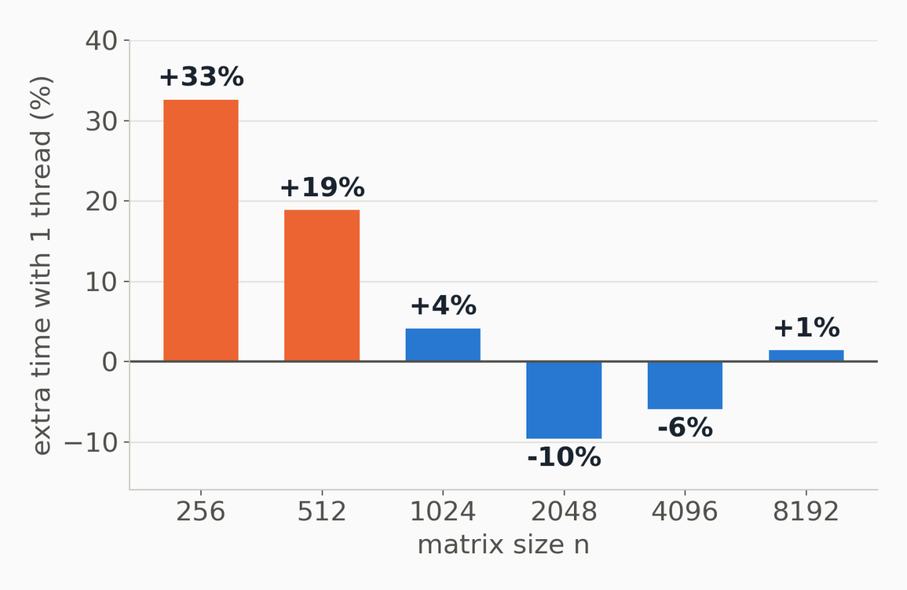
Creating, starting and waiting for a thread takes time, the same for any size.

Time for a 256 × 256 matrix (ms)



Best with 12 threads, only 2.7 times faster. With 32, as slow as the sequential.

Parallel version with only 1 thread

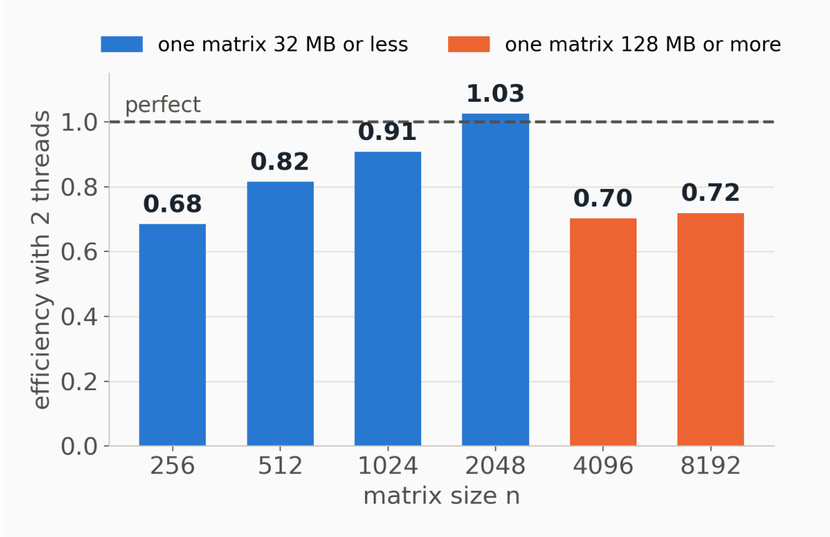


+33% at n = 256. From n = 1024 it is only the normal noise of ±10%.

Bigger matrices share the memory, and conclusions

Efficiency = speedup / threads. 1.0 means every thread works at full speed.

Efficiency with 2 threads



From n = 4096 one matrix (128 MB) does not fit in the 32 MB cache. I think the threads wait for the shared RAM.

× 10 Big matrices with 24 threads.

× 2.7 The best for small matrices: the threads cost too much.

0.7 Efficiency when the matrices do not fit in the cache.

Same Result and memory with any number of threads.

Code, data and report: github.com/Pablo5250/matrix-multiplication